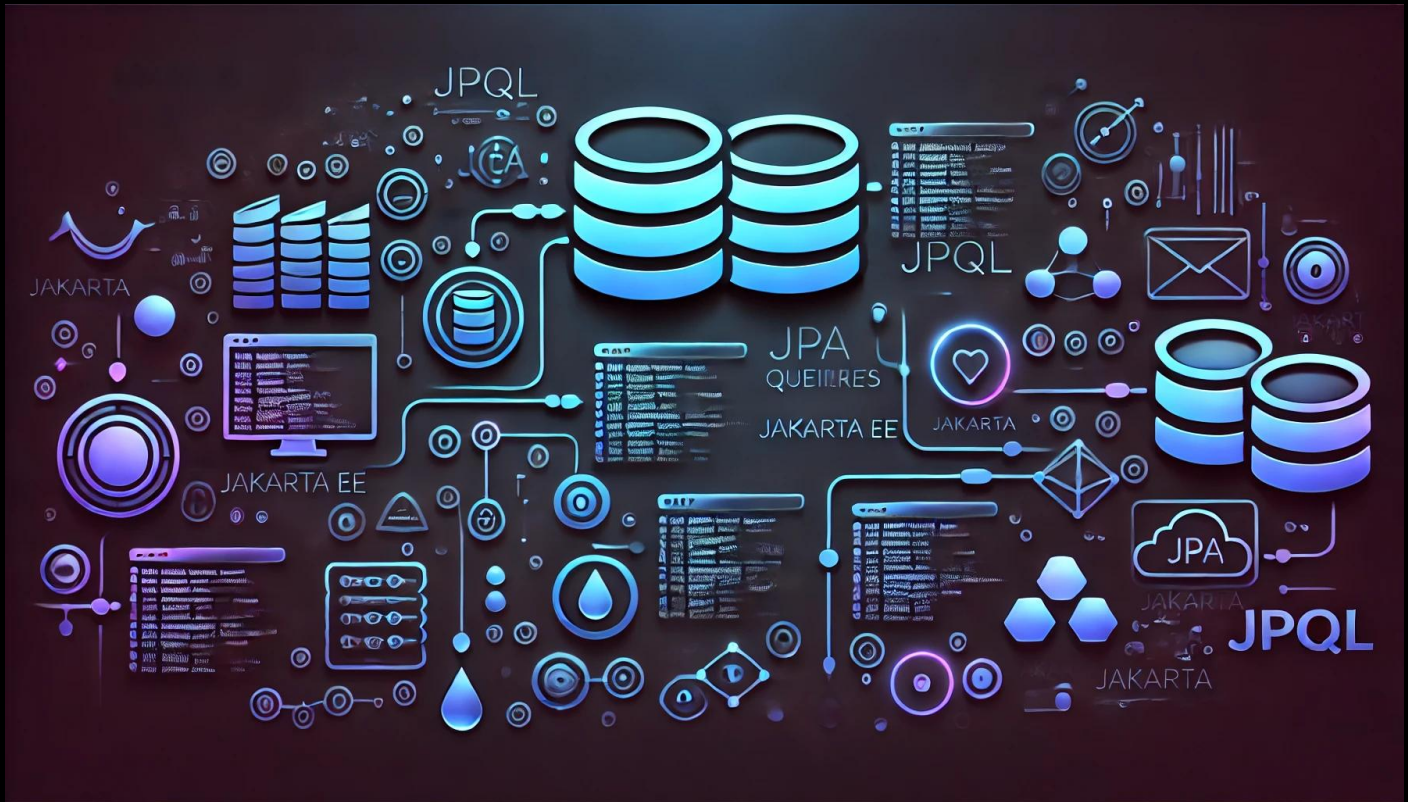


JPQL con JPA y Jakarta EE – parte 7



■ Guía: Consultas con JPQL en JPA - Uso de Parámetros y Filtros con LIKE

◆ Introducción

En esta guía, aprenderemos a utilizar **JPQL en JPA** para realizar consultas dinámicas utilizando **parámetros y filtros con LIKE**. Esto nos permitirá:

1. **Obtener un objeto Persona filtrando por ID, usando parámetros en JPQL.**
2. **Obtener personas que contengan una letra específica en su nombre, sin importar mayúsculas o minúsculas, usando LIKE con UPPER().**

✦ Casos de uso:

- ✓ Filtrar por ID cuando el usuario introduce un valor en un formulario.
- ✓ Buscar registros que contengan una letra específica en cualquier parte del nombre.

1 Conceptos Básicos

✂ ¿Qué son los parámetros en JPQL?

Los **parámetros en JPQL** permiten hacer consultas dinámicas sin necesidad de modificar la consulta directamente. Se usan con el prefijo `:` seguido de un nombre.

Ejemplo en JPQL:

```
SELECT p FROM Persona p WHERE p.idPersona = :id
```

Aquí `:id` es un parámetro que recibirá un valor dinámico.

✂ ¿Cómo buscar coincidencias en un texto con LIKE?

El operador `LIKE` nos permite buscar coincidencias en cadenas de texto. Se usa junto con `%` para indicar cualquier carácter antes o después de la coincidencia.

Ejemplo en JPQL:

```
SELECT p FROM Persona p WHERE p.nombre LIKE :parametro
```

Si `parametro = "%a%"`, la consulta encontrará todos los nombres que contengan la letra "a", en cualquier posición.

Para ignorar mayúsculas y minúsculas, usamos `UPPER()`:

```
SELECT p FROM Persona p WHERE UPPER(p.nombre) LIKE UPPER(:parametro)
```

2 Solución Paso a Paso

🚀 2.1 Actualizar la Clase `PruebaJPQL`

La clase `PruebaJPQL` ejecutará las consultas y mostrará los resultados. Actualizamos la clase `PruebaJPQL` agregando las siguientes consultas:

3 Consulta: Obtener un objeto Persona filtrando por ID

✦ Queremos obtener una Persona específica según el ID proporcionado.

```
//10. Obtiene el objeto persona con id igual al parametro proporcionado
log.info("\nConsulta 10: Obtener objeto Persona por ID");
int idPersona = 1; // Podemos cambiar este valor dinámicamente
// Definir consulta con parámetro
jpql = "SELECT p FROM Persona p WHERE p.idPersona = :id";
Query consulta = em.createQuery(jpql);
consulta.setParameter("id", idPersona);
// Ejecutar consulta y obtener resultado único
persona = (Persona) consulta.getSingleResult();
log.info("Persona encontrada: " + persona);
```

🗨 Explicación:

- `:id` es un **parámetro** que se reemplaza dinámicamente con `idPersona`.
- `setParameter("id", idPersona)`: Se asigna el valor al parámetro `:id`.
- `getSingleResult()`: Como solo esperamos un resultado, usamos este método.

4 Consulta: Obtener personas que contengan una letra en el nombre

✦ Queremos obtener todas las Personas cuyo nombre contenga la letra "a", sin importar mayúsculas o minúsculas.

```
//11. Obtiene las personas que contengan una letra a en el nombre, sin importar mayusculas o minusculas
log.info("\nConsulta 11: Obtener personas con letra 'a' en el nombre (ignorar mayusculas/minusculas)");
// Definir consulta con LIKE y UPPER()
jpql = "SELECT p FROM Persona p WHERE UPPER(p.nombre) LIKE UPPER(:parametro)";
String parametroString = "%a%"; // Busca la letra 'a' en cualquier posición
// Configurar y ejecutar la consulta
consulta = em.createQuery(jpql);
consulta.setParameter("parametro", parametroString);
personas = consulta.getResultList();
// Mostrar resultados
mostrarPersonas(personas);
```

🗨 Explicación:

- `UPPER(p.nombre) LIKE UPPER(:parametro)`:
 - ◆ Convierte el nombre y el parámetro a **mayúsculas** antes de comparar.
 - ◆ Evita problemas de diferencia entre "a" y "A".

- %a%:
 - ◆ % antes y después de "a" permite encontrar la letra **en cualquier parte del nombre**.
-

5 Código completo

Este es el código completo de la clase `PruebaJPQL`:

```
package sga.cliente.jpql;

import java.util.List;
import jakarta.persistence.*;
import java.util.Iterator;
import sga.domain.Persona;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class PruebaJPQL {

    private static final Logger log = LoggerFactory.getLogger(PruebaJPQL.class);

    public static void main(String[] args) {
        String jpql = null;
        List<Persona> personas = null;

        EntityManagerFactory emf = Persistence.createEntityManagerFactory("SgaPU");
        try (EntityManager em = emf.createEntityManager()) {

            //1. Consulta de todos los objetos de tipo Persona
            log.info("\n1. Consulta de todas las Personas");
            jpql = "select p from Persona p";
            personas = em.createQuery(jpql, Persona.class).getResultList();
            mostrarPersonas(personas);

            //2. Consulta de una persona por ID
            log.info("\n2. Consulta de Persona con ID = 1");
            jpql = "SELECT p FROM Persona p WHERE p.idPersona = 1";
            Persona persona = em.createQuery(jpql, Persona.class).getSingleResult();
            log.info("Persona encontrada: " + persona);

            //3. Consulta de una persona por nombre
            log.info("\n3. Consulta de Persona por nombre 'Ivonne'");
            jpql = "SELECT p FROM Persona p WHERE p.nombre = 'Ivonne'";
```

```
personas = em.createQuery(jpql, Persona.class).getResultList();
mostrarPersonas(personas);

//4. Consulta de datos individuales (tupla)
log.info("\n4. Consulta de datos individuales: nombre, apellido, email");

jpql = "SELECT p.nombre, p.apellido, p.email FROM Persona p";
Iterator iter = em.createQuery(jpql).getResultList().iterator();

while (iter.hasNext()) {
    Object[] tupla = (Object[]) iter.next();
    String nombre = (String) tupla[0];
    String apellido = (String) tupla[1];
    String email = (String) tupla[2];

    log.info("Nombre: " + nombre + ", Apellido: " + apellido + ", Email: " + email);
}

//5. Obtiene el objeto Persona y el id, se crea un arreglo de tipo Object con 2 columnas
log.info("\n5. Obtiene el id y el objeto Persona, se crea un arreglo de tipo Object con 2
columnas");
Object[] tupla = null;
jpql = "select p.idPersona, p from Persona p ";
iter = em.createQuery(jpql).getResultList().iterator();
while (iter.hasNext()) {
    tupla = (Object[]) iter.next();
    int idPersona = (int) tupla[0];
    persona = (Persona) tupla[1];
    log.info("ID persona:" + idPersona);
    log.info("Objeto persona:" + persona);
}

//6. Consulta de todas las personas
log.info("\n6. Consulta de los ids de todas las personas");
jpql = "SELECT p.idPersona FROM Persona p";
List<Integer> ids = em.createQuery(jpql, Integer.class).getResultList();
ids.forEach(idPersona -> log.info("ID persona: " + idPersona));

//7. Regresa el valor minimo y maximo del idPersona (scaler result)
log.info("\n7. Regresa el valor minimo y maximo del idPersona (scaler result)");
jpql = "select MIN(p.idPersona) as MinId, MAX(p.idPersona) as MaxId, COUNT(p.idPersona) as
Contador from Persona p";
iter = em.createQuery(jpql).getResultList().iterator();
while (iter.hasNext()) {
    tupla = (Object[]) iter.next();
```

```
Integer idMin = (Integer) tupla[0];
Integer idMax = (Integer) tupla[1];
Long count = (Long) tupla[2];
log.info("idMin:" + idMin + ", idMax:" + idMax + ", count:" + count);
}

//8. Cuenta los nombres de las personas que son distintos
log.info("\n8. Cuenta los nombres de las personas que son distintos");
jpql = "select COUNT(distinct p.nombre) from Persona p";
Long contador = (Long) em.createQuery(jpql).getSingleResult();
log.info("no. de personas con nombre distinto:" + contador);

//9. Concatena y convierte a mayusculas el nombre y apellido
log.info("\n9. Concatena y convierte a mayusculas el nombre y apellido");
jpql = "select CONCAT(p.nombre, ' ', p.apellido) as Nombre from Persona p";
List<String> nombres = em.createQuery(jpql).getResultList();
for (String nombreCompleto : nombres) {
    log.info(nombreCompleto);
}

//10. Obtiene el objeto persona con id igual al parametro proporcionado
log.info("\nConsulta 10: Obtener objeto Persona por ID");
int idPersona = 1; // Podemos cambiar este valor dinámicamente
// Definir consulta con parámetro
jpql = "SELECT p FROM Persona p WHERE p.idPersona = :id";
Query consulta = em.createQuery(jpql);
consulta.setParameter("id", idPersona);
// Ejecutar consulta y obtener resultado único
persona = (Persona) consulta.getSingleResult();
log.info("Persona encontrada: " + persona);

//11. Obtiene las personas que contengan una letra a en el nombre, sin importar si es
mayusculas o minuscula
log.info("\nConsulta 11: Obtener personas con letra 'a' en el nombre (ignorar
mayusculas/minusculas)");
// Definir consulta con LIKE y UPPER()
jpql = "SELECT p FROM Persona p WHERE UPPER(p.nombre) LIKE UPPER(:parametro)";
String parametroString = "%a%"; // Busca la letra 'a' en cualquier posición
// Configurar y ejecutar la consulta
consulta = em.createQuery(jpql);
consulta.setParameter("parametro", parametroString);
personas = consulta.getResultList();
// Mostrar resultados
mostrarPersonas(personas);
}
```

```
}

private static void mostrarPersonas(List<Persona> personas) {
    for (Persona p : personas) {
        log.info("Persona: " + p);
    }
}
}
```

6 Resultado Esperado

Si ejecutamos la clase PruebaJPQL, veremos en la consola:

Consulta 10: Obtener objeto Persona por ID

```
[main] INFO sga.cliente.jpql.PruebaJPQL - Persona encontrada: Persona{idPersona=1, nombre=Rafael,
apellido=Juarez, email=rjuarez@mail.com, telefono=55667788}
```

```
[main] INFO sga.cliente.jpql.PruebaJPQL -
```

Consulta 11: Obtener personas con letra 'a' en el nombre (ignorar mayusculas/minusculas)

```
[EL Fine]: sql: ServerSession(2073640037)--Connection(976358110)--SELECT id_persona, APELLIDO, EMAIL, NOMBRE,
TELEFONO FROM PERSONA WHERE UPPER(NOMBRE) LIKE UPPER(?)
```

```
bind => [%a%]
```

```
[main] INFO sga.cliente.jpql.PruebaJPQL - Persona: Persona{idPersona=1, nombre=Rafael, apellido=Juarez,
email=rjuarez@mail.com, telefono=55667788}
```

```
[main] INFO sga.cliente.jpql.PruebaJPQL - Persona: Persona{idPersona=3, nombre=Maria, apellido=Esparza,
email=mesparza@mail.com, telefono=55112233}
```

Esto confirma que las consultas JPQL están funcionando correctamente 🎯.

7 Conclusión

🎯 En esta guía aprendimos a utilizar **JPQL en JPA** para hacer consultas dinámicas con parámetros y filtros con LIKE.

♦ Puntos clave:

- ✓ Uso de `:parametro` para evitar consultas fijas.
- ✓ Uso de `LIKE` con `%` para encontrar coincidencias en texto.
- ✓ Uso de `UPPER()` para hacer consultas insensibles a mayúsculas.

🚀 ¡Ahora puedes aplicar estas técnicas en tus proyectos!

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)